

Tesla Roadtrip Logger: Cloudflare D1 Database & Data Retrieval

Optional Structured Database via Cloudflare D1

*Cloudflare D1 is a serverless SQL database (built on SQLite) that can serve as our optional structured datastore*¹. Using D1 for this project provides a reliable way to persist and query data (e.g. trip logs, charging sessions, user settings) without managing any infrastructure. Cloudflare D1 is **fully managed and serverless**, meaning we don't have to provision servers or worry about scaling the database manually. It supports familiar SQL queries and **leverages SQLite semantics**, which is convenient for structured data needs¹. D1 became generally available in 2024 and is designed to integrate seamlessly with Cloudflare Workers and Pages, accessible via a binding in our Worker or through HTTP APIs. Key advantages of using D1 include:

- **Persistent Storage:** We can store processed trip data (such as a summary of drives or charges) for quick retrieval, rather than calling external APIs for every request.
- **Relational Queries:** If we need to join or filter data (for example, querying all charging sessions in a specific state or timeframe), SQL support makes it straightforward.
- **Scalability:** D1 is built to scale horizontally. We can create multiple small databases (e.g. per user or per trip) or one large database; Cloudflare handles the replication and scaling behind the scenes²³. There's a generous free tier and usage-based pricing, so we pay only for the storage and queries we use.
- **Integrated with Workers:** Because D1 is a Cloudflare product, we can bind a D1 database to our Cloudflare Worker project easily. Queries are executed within the Worker, minimizing latency (no separate DB server to connect to over the internet).

In our design, using D1 is **optional but recommended**. Initially, we might fetch data directly from the Tesla/Tessie APIs and display results without heavy storage. This reduces complexity for a quick MVP. However, as the project grows (more users or more data), D1 can be enabled to cache data and support advanced features (like user accounts, trip history archive, and analytics). We will set up a D1 database (e.g. via `wrangler d1 create`) and define tables for the data we want to persist. For example, potential tables could include: `drives` (with columns like `drive_id`, `start_time`, `end_time`, `start_location`, `end_location`, `distance`, etc.), `charges` (`session_id`, `timestamp`, `location`, `kWh`, `cost`, etc.), and `users` or `vehicles` (to track multiple vehicles or user preferences).

Why “optional”?: It means the system should work even if the structured DB is not initialized – for instance, we could fetch data on the fly from the API for each request. But having D1 allows us to store results for faster subsequent queries and offline analysis. We will architect the code such that if the D1 binding is present, data gets saved and read from the database; if not, it falls back to live API calls. This approach gives flexibility during early development and allows fast deployment (fewer moving parts), while keeping the path open to add persistence for production.

Fetching Driving and Charging History (June 1, 2025 – Present)

The core dynamic feature is retrieving the Tesla's recent **driving and charging history** starting from June 1, 2025 up to the current date (live data). We plan to leverage the **Tessie API** for this, because Tessie continuously logs vehicle telemetry and makes historical data accessible via its endpoints. Tesla's own official API (the "Tesla Owner API") does **not** provide historical trip data by itself – it only gives the current state of the car. Any history must be collected by polling and storing data over time ⁴. (In fact, community sources confirm that to get location or trip history, one needs to use third-party services like TeslaFi or Tessie which record the data, since Tesla's API won't return past drives directly ⁴.) Tessie's platform, on the other hand, **monitors the Tesla 24/7 and automatically records all driving activity in the cloud** ⁵. This makes it ideal for retrieving a timeline of drives and charges since a given date.

Using the Tessie API: Tessie offers a developer API with endpoints specifically for historical data. Notably, it has endpoints like **"Get Drives"** and **"Get Charges"** which return logged drive sessions and charging sessions respectively ⁶. We will obtain an access token from the user (or a developer token from Tessie) and call these endpoints to fetch the data. The process will be roughly:

- 1. User Authentication to Tessie:** The user would provide a Tessie API access token (which they can generate in their Tessie account). This token authorizes our requests to read their vehicle data. Tessie's API uses a simple Bearer token in the Authorization header or as a query param for auth ⁷ ⁸. *(If the user isn't on Tessie, we might later integrate direct Tesla API streaming – see notes below – but for now Tessie simplifies historical access.)*
- 2. Calling the Drives Endpoint:** We'll make a GET request to Tessie's `Get Drives` endpoint (e.g. `GET https://api.tessie.com/<vehicle_vin>/drives`). We will include query parameters or filters to limit the data from **June 1, 2025 onwards**. Tessie's API likely allows filtering by date or returns recent drives with pagination. If a direct date filter isn't available, we can fetch all drives and then filter in our code by `start_time >= 2025-06-01`. Given Tessie's focus on telemetry, it should be feasible to get all drives in that range in one or a few requests (depending on how much driving was done). Each "drive" record typically includes data like start/end timestamp, duration, distance, start and end coordinates (and possibly address names), and energy used. We will parse these JSON results and possibly store them in D1 for quick reference.
- 3. Calling the Charges Endpoint:** Similarly, we'll call `GET /<vehicle_vin>/charges` to retrieve all charging sessions in the same date range. This will provide details of each charging event (timestamp, location or charger ID, energy added, cost, etc.). Charging history is important for a road trip log (to know where and when the car was charged). Tessie might return home charging and Supercharging sessions, possibly with identifiers for Supercharger stations. We will gather those and filter to June 1, 2025 onward.
- 4. Combining Data:** With drives and charges data fetched, we can integrate them in the app's logic. For example, we might interleave them on a timeline or map (drives between charges, etc.). Initially, a simple approach is to just list or map them chronologically. Because Tessie provides rich data, we could show routes of each drive (Tessie even has a "Get Driving Path" endpoint for detailed GPS traces of a drive ⁶, which we might use to plot the route on a map). For now, a high-level summary

per drive (distance, time, start/end locations) and per charge (location, energy, cost) suffices to document the journey.

5. **Live Data Continuation:** “Live data” implies we also want to show up-to-the-minute information (e.g. if the car is currently driving or just charged). Tessie’s API can be polled for the latest status, or we can use their streaming API for real-time updates ⁹. For simplicity, we might periodically fetch the latest drive or charge status. Cloudflare Workers could even use a Cron Trigger (scheduled event) to fetch new data every X minutes and update the D1 database. This ensures that as the road trip progresses, our stored data stays up to date. Alternatively, whenever the user opens the app/webpage, we fetch any new drives/charges since the last timestamp we have.

Tesla API Integration (Future Expansion): Since our solution is “geared towards Tesla owners,” eventually we may allow users to connect *without* Tessie, using Tesla’s official API directly. To do that, the user would input their Tesla API credentials or token (Tesla now uses an OAuth token system). We’d then use Tesla’s “owner API” endpoints to get the vehicle’s current state and possibly subscribe to the live streaming API for telemetry. However, because Tesla doesn’t provide historical logs, we would have to start logging data ourselves (similar to how Tessie does). This means once a user connects their Tesla, our system would from that point forward record drives and charges in our database (using either periodic polling or the streaming endpoint). Over time, we’d accumulate the same kind of history that Tessie offers. This approach requires more backend effort and storing potentially large volumes of telemetry, so it’s something to plan for long-term. In the immediate term, leveraging Tessie’s existing dataset (with the user’s permission) is the fastest path to get historical data from June 1 onward. It gives a jump-start if the user already has months of data logged with Tessie. (If not, the user could sign up for Tessie and start logging, or we simply can’t retrieve past data that wasn’t recorded – an alternative is TeslaFi or others, but Tessie is already identified for this project.)

In summary, **Task 2 is confirmed:** we will fetch driving and charging history since 2025-06-01 using Tessie’s API. We will design our Cloudflare Worker to query the Tessie endpoints, parse the JSON, and output a friendly summary (and/or store results in D1). This provides the core dataset for the roadtrip app – essentially answering “Where has the car been and charged since the journey started?” in a detailed way.

Secure Credential Management (Wrangler Secrets & Config)

Because our application will interact with external APIs (Tessie or Tesla), we must handle API keys/tokens securely. We will **never hard-code sensitive credentials** in the code or config. Instead, we will use **Cloudflare Worker secrets** for any API tokens or secrets required. Cloudflare Workers allows attaching encrypted values called secrets, which are only accessible at runtime and not visible in the dashboard after setting ¹⁰. For example, we might run `wrangler secret put TESSIE_API_TOKEN` to store our Tessie access token (for a personal account or a general developer token). This ensures the token is encrypted at rest and not exposed in our Git repository or in the client side.

During development, Cloudflare provides a mechanism to use a local `.dev.vars` file to hold secrets and env variables for testing ¹¹ ¹². We will maintain a `.dev.vars` (added to `.gitignore` so it’s never committed ¹³) containing keys like `TESSIE_API_TOKEN="xyz..."` for local testing. In the Wrangler config (`wrangler.toml`), we’ll define these as text bindings or just reference them via `env.TESSIE_API_TOKEN` in code (since Wrangler injects secrets into the `env` object). By keeping a

backup of secrets in a local file (or a secure password manager) and also setting them in Wrangler for production, we ensure **dev and prod are aligned**. In practice, this means if we update a secret (say Tessie token refresh or a new Tesla API key), we update it in both the Cloudflare environment (`wrangler secret put`) and our local `.dev.vars`. This dual approach was explicitly noted to avoid any mismatches between development and production credentials.

Beyond Tessie tokens, any other config like Tesla API client ID/secret (if needed for OAuth), database connection strings (for D1, Cloudflare handles that via binding so no static password needed), or any third-party API keys will all be managed through secrets. This follows Cloudflare's best practices: *"Do not use plaintext environment variables for sensitive info. Use secrets or the Secrets Store"* ¹⁴. By using Wrangler's built-in secret storage, we mitigate the risk of leaking API credentials and comply with security requirements.

Finally, when handling user-specific tokens (for example, if we allow each user to input their own Tessie or Tesla token), we will also treat those with care. In a multi-user scenario, we might encrypt and store tokens in our D1 database, or utilize Cloudflare's forthcoming Secrets Store (account-level secret vault). For now, assuming a single-user or limited user scenario, Wrangler secrets suffice. We'll just need to build forms or CLI inputs for the user to provide their token, then set it in our system securely.

Deployment and Scalability Considerations

One of the goals is to **develop for fast deployment and production using a framework that scales**. Our chosen stack (Cloudflare Workers + D1 + Tessie API) aligns well with that:

- **Fast Deployment:** Cloudflare Workers can be deployed with a single command (`wrangler deploy`), propagating globally within seconds. There's no need to manage servers, which accelerates the development-to-production pipeline. We can iterate quickly and push updates that go live around the world.
- **Built-in Scalability:** The Worker runs on Cloudflare's global edge network by default, meaning as more users use the app, Cloudflare automatically handles scaling out the compute across regions. We don't worry about load balancers or instance counts. The **stateless nature** of the Worker (with state offloaded to D1 or KV) means it can scale horizontally without issues. Cloudflare D1, while currently having a primary region, is also built to handle reasonable query volumes and will eventually support read replicas for global low-latency access ¹⁵. For our initial use (likely low to moderate query rates), D1's performance will be more than sufficient.
- **Low Latency Globally:** Because the logic is at the edge, users across the US (or world) will get fast responses. Data from Tessie's API will have to be fetched (Tessie's servers are cloud-based too), but Cloudflare's network can make those calls efficiently. If needed, we could cache recent results either in D1 or using the Cloudflare cache API to avoid hitting Tessie too frequently. Tessie allows **unlimited vehicle data polling with no extra cost** on their API layer ¹⁶, so we shouldn't hit hard limits, but caching will improve speed and reduce dependency on external calls for every request.
- **Cron Triggers and Background Tasks:** Cloudflare Workers support Scheduled Events (cron triggers). We can set up a schedule (for example, every hour or every day) to automatically fetch the latest drive and charge data and store it. This ensures the database is up-to-date without user intervention. It offloads work to the background, so when a user requests data, we can serve from our D1 cache immediately. This design helps maintain a snappy UI and also spreads out the external API calls to avoid bursts. It's a good practice for scalability and rate-limiting concerns.

In terms of code architecture, we'll use Cloudflare Worker **Modules** (ES Module syntax) for clarity, possibly structuring our code into functions: e.g. `fetchDriveData(sinceDate)` and `fetchChargeData(sinceDate)` that handle calling Tessie and processing results, and `handleRequest` that ties everything together (maybe serving an HTML/JSON response). Cloudflare's environment allows using modern JavaScript, and we might use frameworks or libraries if needed (though for something relatively straightforward, it might not be necessary). We will also ensure to handle errors gracefully – e.g. Tessie API downtime or invalid tokens – and log them (Cloudflare provides a console for Workers).

Future Enhancements and Business Viability

Looking forward, there are several features we plan to add (beyond the initial scope of tasks 1 & 2):

- **Location Parsing for State Visits:** Eventually, we want to parse the GPS coordinates or addresses from the drives to determine which U.S. states (or countries) the car has visited. Tessie's data likely includes either raw coordinates or already-resolved addresses for the start/end of each drive. If coordinates are available, we can use a reverse geocoding service or a simple geo-boundary lookup to map them to states. This would enable features like highlighting a map of states visited on the road trip. Since this is a Tesla-focused app, we could even tap into the Tesla navigation API or use known Supercharger locations (which often include state info). This parsing (task 3 in the original list) can be complex, so we will tackle it after the core data pipeline is stable. It's not needed for initial deployment, but the design will keep it in mind (for example, structuring the data so we can add a "state" field later, or storing location names if Tessie provides them).
- **Photo/Video Integration:** A unique feature we envision is linking the car's journey data with the user's personal photos or videos from the trip. For instance, if the user took photos at various landmarks, we'd like to attach those to the map or timeline of the drive. This likely requires that the photos have location metadata (GPS tags) or at least timestamps. One approach could be: if a photo's timestamp matches the time the car was in a certain state or at a certain charging stop, we associate it with that segment of the trip. We might allow the user to upload photos or connect their Google Photos, etc., then do a smart match. This is a stretch goal and depends on accurate metadata, as the user noted (we'd need GPS EXIF data in the photos or manual tagging). It's an idea that can add a lot of personal value to the trip log, setting our platform apart from generic data loggers.
- **Multi-User Platform & Monetization:** Currently, we're building this for a specific use-case (possibly the user's own road trip). But there is potential to make it a platform or app for other Tesla owners, especially roadtrip enthusiasts. If no similar solution exists yet (combining automatic drive data logging, state-by-state progress, and photo journaling), we could offer this as a service. Competitors like TeslaFi or Tessie themselves provide raw data and maps, but a focused "roadtrip diary" app could carve out a niche. To prepare for this, we'll ensure the architecture can handle multiple users. That means:
 - Each user can auth with their own Tesla/Tessie credentials (we'll need a secure way to store multiple tokens – possibly using D1 with encryption, or leveraging OAuth flows).
 - Data separation: use a `user_id` or similar key in the database so each user only accesses their own trip data.

- Perhaps build a simple frontend where users sign up/log in (could be as simple as GitHub OAuth or email login for our app, then they input Tesla API token). Cloudflare Workers can serve the frontend (or Cloudflare Pages for static UI).
- Scalability for many users is already handled by Cloudflare's serverless nature, but we'd watch out for Tessie API rate limits or pricing if many accounts poll data. Tessie might charge for heavy usage or require a higher plan for API access if scaled out; we'd need to investigate Tessie's terms for commercial use.
- **Tesla API Direct Enhancements:** As mentioned, adding direct Tesla API support would broaden our user base (not requiring a Tessie subscription). Tesla's API is unofficial but widely used; however, Tesla sometimes changes access rules (e.g., recent changes requiring 3rd-party apps to use an official API proxy or new permissions). We'll keep an eye on Tesla's developer landscape. Services like Smartcar or Tesla's own "*Fleet API*" might provide official data access for third-party apps ¹⁷ ¹⁸. Tessie's developer site even mentions a "Tesla Fleet API layer" and "Fleet Telemetry streaming API" they provide ¹⁶ – possibly we could partner with Tessie or use their infrastructure under the hood for non-Tessie users as well, to avoid reinventing the wheel. These are strategic decisions for later; initially, using Tessie directly is the fastest path.

In conclusion, focusing on tasks **1 & 2** sets up the foundation: **a robust, scalable data backend (Cloudflare D1)** and **a reliable data ingestion from Tesla's history (via Tessie)**. By adhering to best practices (secure secrets management, efficient use of serverless tech) we ensure the project is production-ready and maintainable. The envisioned product could fill a gap for Tesla owners who want a rich record of their travels. With incremental additions (state tracking, media, social sharing perhaps), this could indeed be packaged as an app for roadtrippers and travelers, potentially turning into a commercial offering if we confirm that no one else already provides the same experience. The combination of Tesla's vehicle data and personal travel memories is compelling, and our implementation is on track to deliver that with modern, scalable technology.

Sources:

1. Cloudflare D1 documentation – Cloudflare's serverless SQLite-based database (features and overview) ¹ ³
2. Cloudflare Workers secrets – Best practice for storing API keys (encrypted, not in repo) ¹⁰ ¹⁹
3. Tessie Help Center – Tessie logs all driving activity 24/7 for later access ⁵
4. Reddit Q&A – Tesla's API only provides current state; historical trip data requires third-party logging (e.g. TeslaFi/Tessie) ⁴
5. Tessie Developer API reference – Endpoints exist to fetch drives and charges history programmatically ⁶

¹ ¹⁵ Building D1: a Global Database
<https://blog.cloudflare.com/building-d1-a-global-database/>

² ³ Overview · Cloudflare D1 docs
<https://developers.cloudflare.com/d1/>

4 Can you get location history data via the Tesla API? : r/TeslaSupport

https://www.reddit.com/r/TeslaSupport/comments/16lctxb9/can_you_get_location_history_data_via_the_tesla/

5 Drive Tracking - Help Center - Tessie

<https://help.tessie.com/article/43-drive-tracking>

6 7 9 16 Compare APIs - Tessie

<https://developer.tessie.com/reference/tesla-api-comparison>

8 Access the Tessie API

<https://developer.tessie.com/reference/overview>

10 11 12 Secrets · Cloudflare Workers docs

<https://developers.cloudflare.com/workers/configuration/secrets/>

13 19 Environment variables and secrets · Cloudflare Workers docs

<https://developers.cloudflare.com/workers/development-testing/environment-variables/>

14 Environment variables - Workers - Cloudflare Docs

<https://developers.cloudflare.com/workers/configuration/environment-variables/>

17 Tesla APIs made easy - Tessie

<https://tessie.com/developers>

18 Tessie: The Tesla management platform

<https://tessie.com/>